

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

DEPENDENCIAS FUNCIONALES Y NORMALIZACIÓN

Clase 3 · Taller T8002 — Representación de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8002 — Representación de datos

Cohorte o comisión: Cohorte 2026

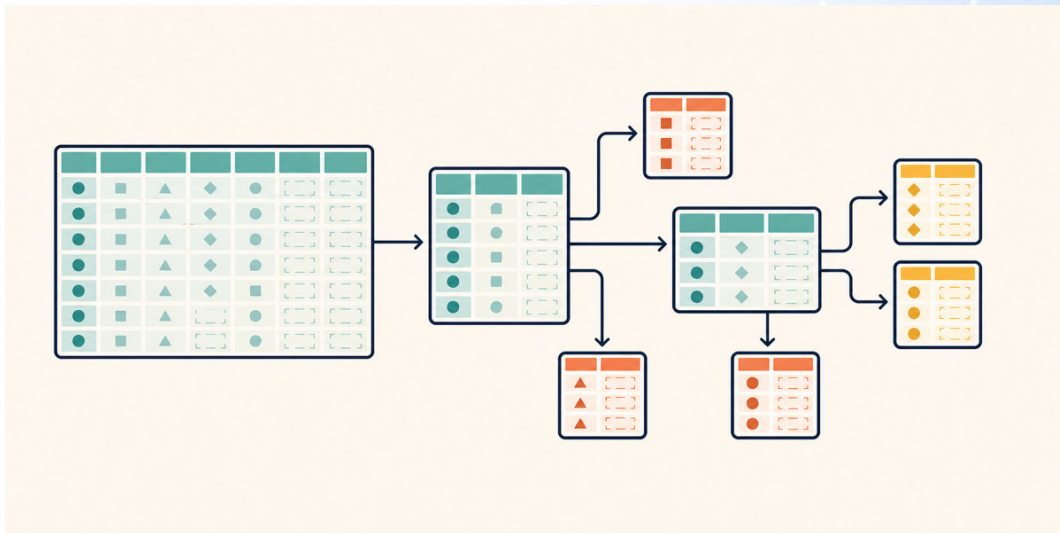


Figura 1. La misma separación de la clase 2, ahora en tres pasos que se pueden verificar uno por uno.

1. INTRODUCCIÓN

1.1 Lo que la clase 2 dejó anotado

La clase 2 llegó al diseño de seis tablas y lo hizo **a ojo**. El método era razonable: buscar columnas que se repitan siempre juntas, preguntar si eso que describen tiene propiedades propias, dibujar las cajas y pasar a tablas.

Funcionó porque el relevamiento es chico y porque las entidades eran evidentes. Con doce columnas y tres localidades, cualquiera ve que la provincia va aparte.

El problema es qué pasa con una tabla de cuarenta columnas donde nada es evidente. Ahí el ojo falla, y no falla avisando: falla dejando una repetición adentro que nadie nota hasta que aparece la primera contradicción.

Esta clase es el método formal. Hace tres cosas:

1. Le pone nombre y notación a "columnas que se repiten siempre juntas": es una **dependencia funcional**.
2. Da un procedimiento de tres pasos que se puede seguir sin intuición, y que termina cuando no queda nada por separar: son las **formas normales**.
3. **Verifica el diseño de la clase 2**. Al final de la sección 4 vamos a llegar, por el camino formal, a las mismas seis tablas. Si llegáramos a otras, una de las dos clases estaría mal.

1.2 Lo que se supone de las clases anteriores

De la clase 1: **clave primaria**, **clave compuesta**, **clave foránea** y **nulo**.

De la clase 2: **entidad, cardinalidad, el cruce de tablas** y el esquema de seis tablas de su sección 4.4. El cruce es el que más falta va a hacer, porque es la prueba de que una separación está bien hecha.

Del T8001, una sola cosa y es la que abre la sección 4: la **regla cardinal de los datos tabulares**, de la sección 3.5 de su clase 1.

1.3 La tabla plana de esta clase

La misma de siempre: relevamiento_plano.csv, 45 filas, 12 columnas, 37 hogares.

Un aviso de vocabulario, porque en el área se dicen de todo para nombrar lo mismo. Esta tabla es una **tabla plana**: una sola tabla que guarda todo. La vas a encontrar llamada tabla única, tabla ancha, tabla desnormalizada, y en las planillas de trabajo real "la planilla". En este apunte se dice **tabla plana** para la tabla del relevamiento. La sección 5 usa además **tabla ancha**, en el sentido de formato que traía el T8001, y lo avisa ahí.

Estas son cinco de sus filas, con las columnas que vamos a usar todo el día:

id_hogar	fecha_visita	localidad	provincia	encuestador_legajo	encuestador_nombre	personas	servicios
1	2025-03-14	Ramallo	Buenos Aires	E04	Sosa, Diego	3	agua;luz;gas
1	2025-06-12	Ramallo	Buenos Aires	E04	Sosa, Diego	4	agua;luz;gas
2	2025-03-15	Córdoba	Córdoba	E02	Ledesma, Julio	5	agua;luz
3	2025-03-15	Concepción	Tucumán	E03	Bianchi, Ana	2	luz
4	2025-03-16	Ramallo	Buenos Aires	E01	Rivas, Marta	4	agua;luz;gas

Mirá las dos filas del hogar 1 y compará columna por columna. personas cambió, de 3 a 4. localidad y provincia no cambiaron, y no podían cambiar. Esa diferencia entre dos columnas de la misma tabla es todo el contenido de la clase.

2. DEPENDENCIAS FUNCIONALES

2.1 Qué quiere decir que un atributo determina a otro

La idea es más simple que su nombre. Una columna **determina** a otra si conocer el valor de la primera alcanza para saber el valor de la segunda.

Si te digo que la localidad es Ramallo, ¿podés decirme la provincia? Sí: Buenos Aires. Siempre, sin mirar el resto de la fila. Entonces **localidad determina provincia**.

Si te digo que la localidad es Ramallo, ¿podés decirme cuántas personas hay en el hogar? No. Hay veinte visitas a hogares de Ramallo y tienen distinta cantidad de personas. Entonces **localidad no determina personas**.

Eso es una **dependencia funcional**, y la prueba es siempre la misma pregunta: *dado este valor, ¿el otro queda fijado?*

Dos precisiones que evitan la mayoría de los errores:

- **Es en un solo sentido.** localidad determina provincia, pero provincia no determina localidad: la provincia de Córdoba podría tener varias localidades relevadas. Que hoy tenga una sola es una casualidad de estos datos.
- **No es causalidad.** No es que Ramallo *cause* Buenos Aires. Es que en este universo de datos, cada localidad tiene exactamente una provincia.

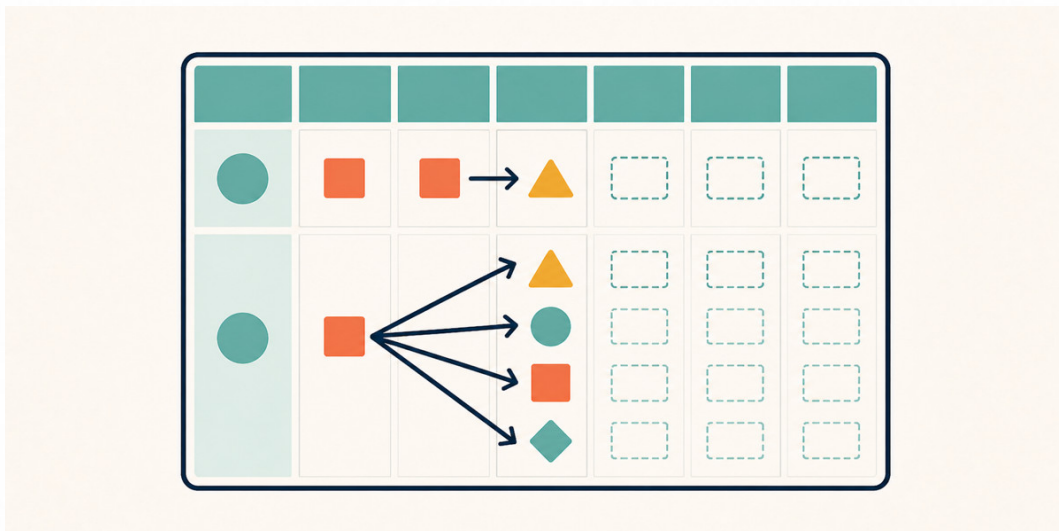


Figura 2. La prueba es una sola pregunta: dado este valor, ¿el otro queda fijado?

2.2 La notación

Se escribe con una flecha, y se lee "determina":

localidad → provincia

Se lee: *localidad determina provincia*. Del lado izquierdo va lo que se conoce, y del derecho lo que queda fijado.

A la izquierda puede haber más de una columna, y entonces se separan con coma. Eso quiere decir que hacen falta **las dos juntas** para fijar el valor:

```
id_hogar, fecha_visita → personas
```

Se lee: *el hogar y la fecha, juntos, determinan la cantidad de personas*. Y es cierto: el hogar 1 el 14 de marzo tenía 3 personas. Ni el hogar solo ni la fecha sola alcanzan.

Y una notación más, para agrupar varias flechas que salen del mismo lado izquierdo:

```
encuestador_legajo → encuestador_nombre, encuestador_telefono
```

Una flecha es una columna del lado derecho. Ese renglón de arriba son dos flechas, no una: el legajo determina el nombre, y el legajo determina el teléfono. Se escriben juntas porque comparten el lado izquierdo. Vale la aclaración porque en la sección 2.6 vamos a contarlas, y contar renglones da un número distinto de contar flechas.

2.3 Cómo se busca una dependencia funcional en los datos

La búsqueda a ojo funciona con doce columnas y se vuelve imposible con cuarenta. El procedimiento mecánico es este, y es el que hace el notebook:

Para cada par de columnas A y B, agrupá las filas por A y contá cuántos valores distintos de B hay en cada grupo. Si en todos los grupos hay exactamente uno, entonces $A \rightarrow B$.

Aplicado a *localidad* y *provincia*:

Grupo (localidad)	Filas	Valores distintos de provincia
Ramallo	20	1 (Buenos Aires)
Córdoba	12	1 (Córdoba)
Concepción	13	1 (Tucumán)

Todos los grupos tienen exactamente un valor. Entonces *localidad* $\rightarrow$ *provincia*.

Aplicado a *localidad* y *personas*:

Grupo (localidad)	Filas	Valores distintos de personas
Ramallo	20	9
Córdoba	12	6
Concepción	13	6

Hay grupos con más de un valor. Entonces *localidad* no determina *personas*.

2.4 La dependencia que los datos sugieren y la regla no admite

Acá está la trampa más importante de la clase, y es la misma de la cardinalidad en la clase 2.

El procedimiento de la sección anterior no encuentra dependencias funcionales. Encuentra candidatas. Con 45 filas, varias columnas pasan la prueba por casualidad.

Dos casos reales de esta tabla:

Caso 1: provincia → localidad **pasa la prueba, y es falso.** Agrupá por provincia y contá localidades: Buenos Aires tiene una sola, Ramallo. Córdoba tiene una sola. Tucumán tiene una sola. Todos los grupos tienen un valor, así que el procedimiento dice que sí. Y es falso: la provincia de Buenos Aires tiene miles de localidades, y el día que el relevamiento incorpore Pergamino la dependencia se cae. Lo que pasó es que este relevamiento tiene una localidad por provincia.

Caso 2: encuestador_legajo → localidad **pasa la prueba en tres de cuatro casos.** E01 solo trabajó en Ramallo, E02 y E03 en una localidad cada uno. Si E04 no existiera, el procedimiento diría que el encuestador determina la localidad. E04 trabajó en Ramallo y en Córdoba, y por una sola persona la dependencia se cae. Y ojo: aunque E04 no existiera, la dependencia seguiría siendo falsa, porque nada en el operativo impide que un encuestador cubra dos localidades.

La regla, entonces:

*Los datos que tenés **descartan** dependencias funcionales: un solo contraejemplo alcanza para matar una. Lo que no pueden hacer es **confirmarlas**. Una dependencia funcional es una regla del negocio, y se confirma preguntándole a quien conoce el negocio.*

Es la misma lección que la clase 2 dio sobre la cardinalidad, y no es casualidad: son la misma cosa mirada de dos maneras.

2.5 Cuando la dependencia es de una parte de la clave, y cuando es en cadena

De todas las dependencias funcionales de una tabla, hay tres formas que importan, y se distinguen por dónde está el lado izquierdo respecto de la clave. La clave del relevamiento es el par id_hogar, fecha_visita.

Total: el lado izquierdo es la clave entera.

id_hogar, fecha_visita → personas

Esta está bien. Es lo que se espera de una tabla: los atributos dependen de la clave completa.

Parcial: el lado izquierdo es una parte de la clave.

id_hogar → localidad

id_hogar es la mitad de la clave. La localidad de un hogar no cambia con la fecha, así que la mitad de la clave alcanza para fijarla. Esta es la que hace que la localidad se repita en las dos filas del hogar 1, y es el defecto que arregla la segunda forma normal.

Transitiva: el lado izquierdo no es parte de la clave.

localidad → provincia

localidad no es clave ni parte de la clave: es un atributo cualquiera. Y determina a otro. Se llama transitiva porque forma una cadena: la clave determina la localidad, y la localidad determina la provincia. Es el defecto que arregla la tercera forma normal.

Forma	Lado izquierdo	Ejemplo	La arregla
total	la clave entera	id_hogar, fecha_visita → personas	nada, está bien
parcial	parte de la clave	id_hogar → localidad	la 2FN
transitiva	un atributo que no es clave	localidad → provincia	la 3FN

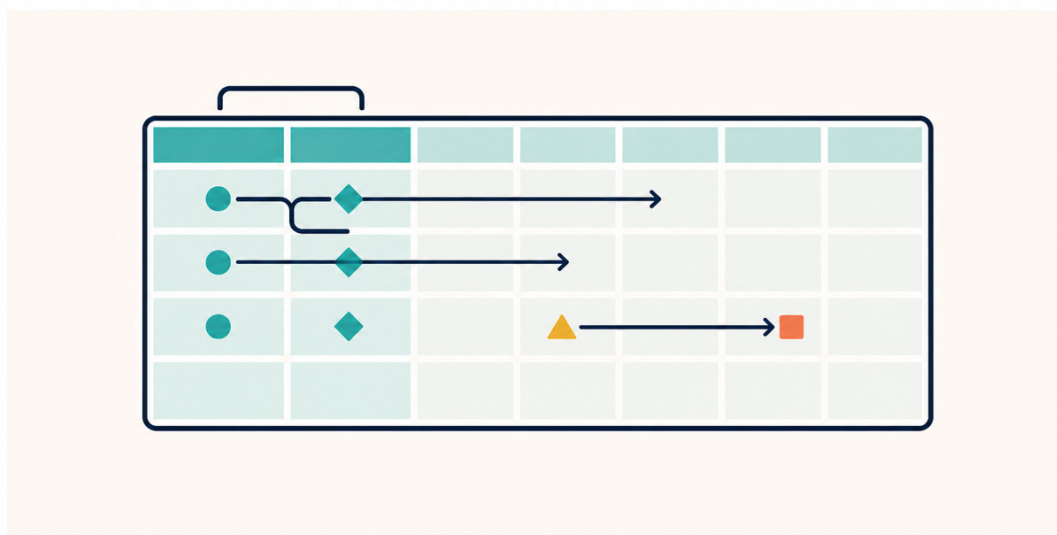


Figura 3. De dónde sale la flecha decide qué forma normal se está violando.

Si la clave es de una sola columna, la dependencia parcial no puede existir, porque no hay "parte" de una columna sola. Acá la clave es compuesta, y por eso hay caso. La sección 4.3 vuelve sobre esto.

2.6 Las dependencias funcionales del relevamiento

La lista completa. Son **diez flechas**, escritas en nueve renglones, agrupadas por forma:

Totales (dependen de la clave entera, y están bien)

```
id_hogar, fecha_visita → encuestador_legajo
id_hogar, fecha_visita → personas
id_hogar, fecha_visita → ingreso
id_hogar, fecha_visita → cobertura_salud
id_hogar, fecha_visita → satisfaccion
```

Parciales (dependen de id_hogar, que es media clave)

```
id_hogar → localidad
id_hogar → servicios
```

Transitivas (el lado izquierdo no es clave ni parte de clave)

localidad → provincia

encuestador_legajo → encuestador_nombre, encuestador_telefono

Diez flechas en total: cinco totales, dos parciales y tres transitivas. Y ahora leelas de nuevo pensando en las anomalías de la clase 1: cada dependencia que **no** es total es exactamente un hecho que la tabla está escribiendo más veces de las necesarias.

- id_hogar → localidad: por eso la localidad del hogar 1 está dos veces.
- localidad → provincia: por eso el par Ramallo — Buenos Aires está veinte veces.
- encuestador_legajo → encuestador_telefono: por eso el teléfono de E04 está diecisiete veces.

Las cinco dependencias que no son totales son la redundancia, escrita en notación. Eso es lo que hace útil todo este aparato: convierte "acá hay algo repetido" en una lista finita de cosas concretas que se pueden arreglar de a una.

3. NORMALIZAR

3.1 Una palabra que ya significaba tres cosas

Antes de seguir hay que desactivar una confusión, porque el T8001 la dejó armada.

El T8001, en su clase 3, avisó que la palabra **normalización** nombra operaciones distintas que no tienen nada que ver entre sí. Tres, para ser exactos:

1. **Codificar** una variable: convertir mala, regular, buena en números, o abrir una nominal en una columna por categoría.
2. **Escalar** una variable numérica: llevarla a un rango común con min-max, o centrarla con z-score.
3. **Normalizar la estructura** de una tabla: pasar de formato ancho a formato largo, y al revés.

Este taller agrega **una cuarta**, y es la de hoy: normalizar en el sentido de las **formas normales**, que es separar una tabla en varias para que cada hecho quede escrito una sola vez.

La cuarta no tiene ninguna relación con las tres primeras. Y ojo con la tercera, que es la que más se parece y no es: pasar de ancho a largo es cambiar cómo se acomodan los mismos valores en una tabla sola. Las formas normales **crean tablas nuevas** y establecen referencias entre ellas.

Cuando alguien dice "hay que normalizar", en este taller quiere decir la cuarta. En una reunión, preguntá cuál.

3.2 Qué problema resuelve

Uno solo, y ya lo vimos entero: **la redundancia y las tres anomalías que produce.**

Normalizar es el procedimiento que retira redundancia de manera controlada. Y controlada quiere decir dos cosas:

- Es **sistemático**: hay pasos, y cada paso tiene un criterio para saber si se cumplió.
- Es **reversible**: cruzando las tablas resultantes se recupera la tabla original, exactamente.

La segunda es la que lo vuelve confiable. Sin ella, separar tablas sería una apuesta.

3.3 Descomponer sin perder información

Separar una tabla en dos es fácil. Separarla bien no.

Una descomposición es **buena** si al cruzar las dos tablas resultantes se recupera la tabla original: las mismas filas, ni una más ni una menos. Si aparecen filas que no estaban, la descomposición perdió información, y perdió justamente la que decía qué iba con qué.

Ejemplo con tres filas, para verlo. Partimos de esto:

id_hogar	localidad	personas
1	Ramallo	3
2	Córdoba	5
4	Ramallo	4

Descomposición buena. Cortamos dejando id_hogar en las dos tablas:

id_hogar	localidad		id_hogar	personas
1	Ramallo		1	3
2	Córdoba		2	5
4	Ramallo		4	4

Cruzando por id_hogar vuelven las tres filas originales. Nada se perdió.

Descomposición mala. Cortamos sin dejar nada en común:

id_hogar	personas		localidad
1	3		Ramallo
2	5		Córdoba
4	4		

Ahora no hay forma de saber que el hogar 1 estaba en Ramallo. Cruzando estas dos tablas sale cada hogar con cada localidad: seis filas en lugar de tres, y tres son inventadas. La tabla de localidades tiene dos valores distintos y no tres, porque una tabla del modelo relacional no tiene filas repetidas.

De ahí sale la regla operativa, y es la única que hay que recordar:

Al separar una tabla en dos, la columna que las une tiene que quedar en las dos. Esa columna es la clave de una de ellas, y en la otra queda como clave foránea. Es exactamente lo que hace la regla 2 del pasaje de la clase 2.

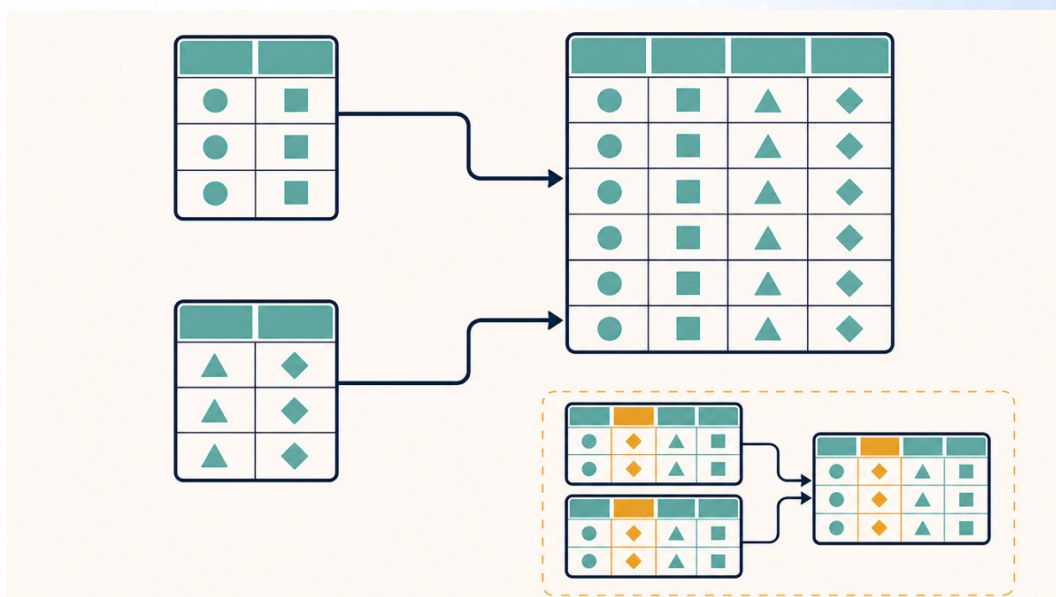


Figura 4. Sin columna compartida, el cruce no reconstruye: multiplica. Y las filas que inventa parecen datos.

3.4 Las tres formas, y por qué son acumulativas

Las formas normales son niveles. Cada uno retira un tipo distinto de repetición, y cada uno supone el anterior: una tabla está en tercera forma normal solamente si además está en segunda y en primera.

El anticipo, un renglón cada una:

Forma	Qué exige	Qué defecto retira
1FN	un valor por celda	la celda con varios valores
2FN	1FN, y ningún atributo depende de una parte de la clave	la dependencia parcial
3FN	2FN, y ningún atributo depende de otro que no sea clave	la dependencia transitiva

Que sean acumulativas tiene una consecuencia práctica: **hay que hacerlas en orden**. Buscar dependencias parciales en una tabla con celdas multivalor no tiene sentido, porque todavía no se sabe cuál es la clave de verdad.

4. LAS TRES FORMAS NORMALES

4.1 Primera forma normal: esta ya la sabés

Una tabla está en **primera forma normal** si cada celda tiene un solo valor, y si cada columna guarda una sola cosa.

Y acá va la buena noticia del día: **eso ya lo aprendiste en el T8001, con otro nombre**. La regla cardinal de los datos tabulares, de la sección 3.5 de la clase 1 de ese taller, decía:

1. Una variable por columna.
2. Una observación por fila.
3. Un valor por celda.

Las condiciones 1 y 3 **son** la primera forma normal. La condición 2 es la que la clase 1 de este taller volvió a mirar cuando descubrió que una fila del relevamiento es una visita y no un hogar.

Así que la 1FN sale gratis, y conviene decirlo con claridad para no perder tiempo: si el T8001 dejó la tabla cumpliendo su regla cardinal, ya está en primera forma normal. Lo único que quedó afuera es servicios, porque ese taller lo abrió para poder contar y después volvió a guardar la columna original.

Eso compra tiempo, y hace falta: la segunda y la tercera forma son lo nuevo, y son las difíciles.

4.2 De arreglo de formato a tabla propia

La columna servicios del relevamiento guarda agua;luz;gas. Cuarta vez que este taller y el anterior se topan con ella, así que corresponde decir qué es distinto ahora.

- El T8001, en su clase 2, la **contó**: descubrió seis combinaciones distintas para cuatro servicios y la marcó como problema.
- El T8001, en su clase 3, la **abrió**: la partió en columnas para poder contar cuántos hogares tenían cada servicio. Era un arreglo de formato, hecho para un cálculo puntual, sobre una copia. La columna original quedó como estaba.
- La clase 2 de este taller, en su sección 4.3, la **vinculó**: reconoció que hogar y servicio es un vínculo muchos a muchos, y de ahí salió hogar_servicio.
- Hoy se convierte en una **tabla propia**, con su clave y su clave foránea. No es un arreglo para una cuenta: es el lugar definitivo donde ese dato vive.

La diferencia importa. Abrir la columna en cuatro columnas tiene_agua, tiene_luz, tiene_gas, tiene_cloacas resuelve la 1FN y crea un problema nuevo: el día que aparezca un quinto servicio hay que **cambiar la estructura de la tabla**, y todo lo que la consulte. Con una tabla propia, un servicio nuevo es una fila nueva.

El resultado son las dos tablas que la clase 2 ya había obtenido por el camino conceptual:

servicio — 4 filas

id	nombre
1	agua
2	luz
3	gas
4	cloacas

hogar_servicio — 88 filas, clave compuesta por las dos columnas

hogar_id	servicio_id
1	1
1	2
1	3
2	1
2	2

Cómo se reconoce una violación de 1FN. Buscá celdas con separadores adentro: ;, ,, /, barras, o texto con saltos de línea. Y buscá columnas que se llamen en plural. servicios, telefonos, observaciones: el plural en el nombre de una columna es casi siempre el rastro de una tabla que falta.

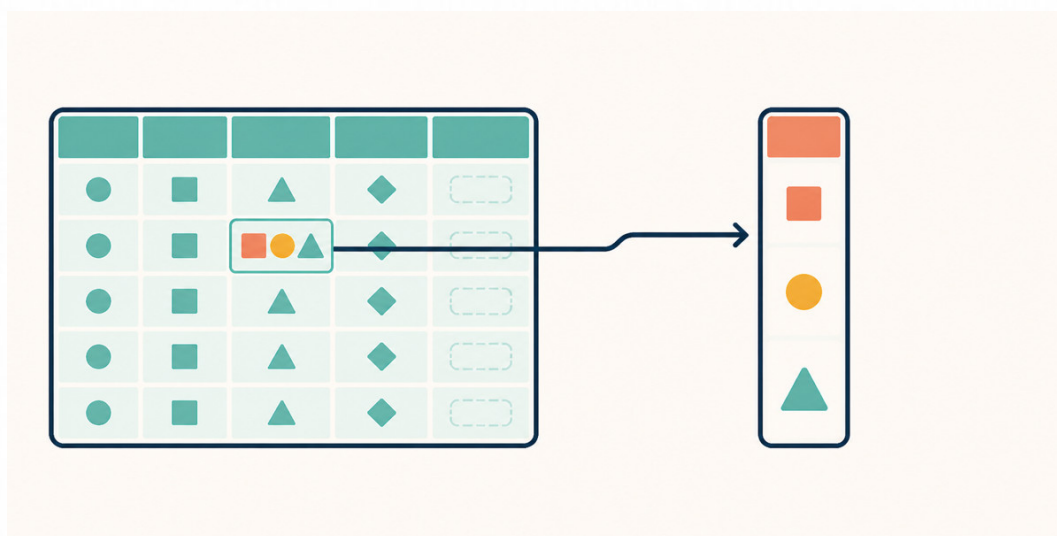


Figura 5. Cada valor de la celda se vuelve una fila. Y un valor nuevo ya no obliga a cambiar la estructura.

4.3 Segunda forma normal: nada depende de una parte de la clave

Una tabla está en **segunda forma normal** si está en 1FN y además **ningún atributo depende de una parte de la clave**. O sea: si no tiene dependencias parciales.

En el relevamiento la clave es `id_hogar`, `fecha_visita`, y hay dos dependencias parciales:

`id_hogar` → `localidad`

`id_hogar` → `servicios`

`servicios` ya salió en el paso anterior. Queda `localidad`, y el síntoma es visible: en las dos filas del hogar 1, la localidad está escrita dos veces. Cada visita adicional a un hogar es una copia más de su localidad.

El arreglo. Los atributos que dependen de una parte de la clave se van a una tabla nueva, cuya clave es justamente esa parte:

hogar — 37 filas, clave `id`

id	localidad	provincia
1	Ramallo	Buenos Aires
2	Córdoba	Córdoba
3	Concepción	Tucumán
4	Ramallo	Buenos Aires

`provincia` se va con `localidad`, porque también depende solo de `id_hogar`. Que siga repetida acá es justamente el defecto que arregla el paso siguiente.

visita — 45 filas, clave `hogar_id`, `fecha`, recortada a seis columnas

hogar_id	fecha	encuestador_legajo	encuestador_nombre	personas	ingreso
1	2025-03-14	E04	Sosa, Diego	3	185000
1	2025-06-12	E04	Sosa, Diego	4	201000
2	2025-03-15	E02	Ledesma, Julio	5	240500

La localidad del hogar 1 quedó escrita **una vez**. Y notá que la localidad no desapareció de la tabla `visita` por casualidad: se fue porque no era un dato de la visita. Nunca lo fue.

La 2FN solo tiene sentido con clave compuesta. Si la clave es de una columna, no hay partes, y la tabla está en 2FN automáticamente. Cuando alguien dice que la 2FN "casi nunca hace falta", está hablando de bases con claves sustitutas de una sola columna, donde es cierto.

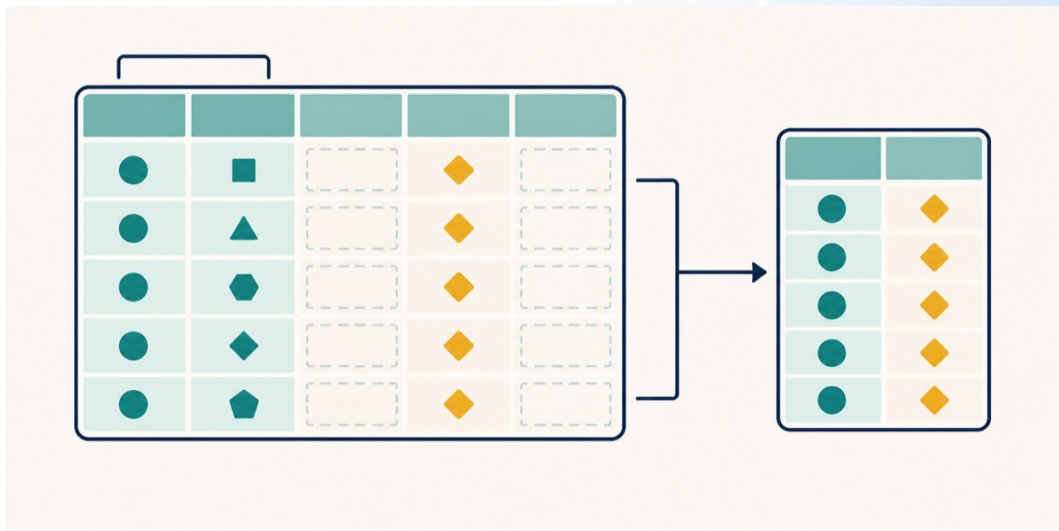


Figura 6. Si un dato no cambia cuando cambia media clave, es que pertenece a la otra media.

4.4 Tercera forma normal: nada depende de un atributo que no es clave

Una tabla está en **tercera forma normal** si está en 2FN y además **ningún atributo depende de otro atributo que no sea clave**. O sea: si no tiene dependencias transitivas.

Quedaron tres, escritas en dos renglones, y están en las dos tablas del paso anterior:

```
en `hogar`:    localidad → provincia
en `visita`:   encuestador_legajo → encuestador_nombre, encuestador_telefono
```

Las dos tienen la misma forma: la clave determina un atributo, y ese atributo determina otro. hogar.id determina localidad, y localidad determina provincia. Una cadena de dos pasos, y de ahí el nombre.

El síntoma es el de siempre: el par Ramallo — Buenos Aires escrito veinte veces, el teléfono de E04 escrito diecisiete.

El arreglo. El atributo del medio de la cadena se lleva a una tabla nueva, donde pasa a ser clave, y en la tabla original queda una referencia:

localidad — 3 filas

id	nombre	provincia
1	Ramallo	Buenos Aires
2	Córdoba	Córdoba
3	Concepción	Tucumán

hogar — 37 filas

id	localidad_id
1	1
2	2
3	3
4	1

encuestador — 4 filas

legajo	nombre	telefono
E01	Rivas, Marta	3407-412233
E02	Ledesma, Julio	351-4778120
E03	Bianchi, Ana	381-4551907
E04	Sosa, Diego	3407-419854

visita — 45 filas, con encuestador_legajo como clave foránea y sin las columnas del nombre y el teléfono.

Ahora el teléfono de E04 está en una celda. Cambiarlo es cambiar una celda, y la anomalía de modificación de la clase 1 dejó de existir. No se mitigó: **dejó de ser posible**.

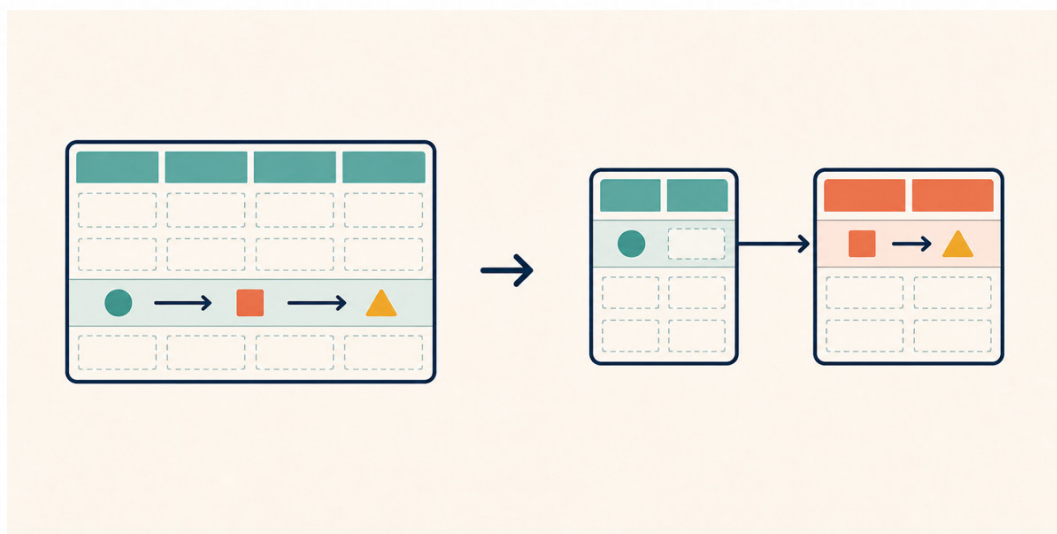


Figura 7. Cuando la clave llega al tercer dato pasando por el segundo, el segundo es una tabla.

4.5 Por qué el taller para en la tercera

Hay más formas normales. Existen la forma normal de Boyce-Codd, la cuarta y la quinta, y cada una retira un tipo de repetición más raro que la anterior.

Este taller para en la tercera por dos razones, y las dos son honestas:

- **Con la tercera alcanza para casi todo.** En diseños de gestión, una tabla en 3FN está bien diseñada. Los casos que la 3FN no cubre son poco frecuentes y bastante artificiales.
- **Las que siguen son contenido de Modelado de datos.** Ahí se ven con el aparato completo, y además con la práctica de implementarlas.

Si te cruzás con la sigla BCNF en un texto, es una versión un poco más estricta de la tercera. No hace falta hoy.

4.6 El relevamiento, paso a paso, y la verificación del esquema de la clase 2

Ahora el recorrido entero, y con un propósito preciso: **comprobar que el camino formal llega al mismo lugar que el camino conceptual de la clase 2.**

Punto de partida. Una tabla plana, 45 filas, 12 columnas. Clave: `id_hogar`, `fecha_visita`. Diez dependencias funcionales: cinco totales, dos parciales y tres transitivas.

Paso 1 — a primera forma normal. El único defecto es `servicios`, que guarda hasta cuatro valores en una celda. Sale a dos tablas: `servicio` con los cuatro nombres, y `hogar_servicio` con 88 filas.

Quedan: la tabla plana sin `servicios`, más `servicio`, más `hogar_servicio`. **Tres tablas.**

Paso 2 — a segunda forma normal. Queda una dependencia parcial: `id_hogar` → `localidad`. Los atributos que dependen solo de `id_hogar` se van a una tabla `hogar` con clave `id`, y el resto queda en `visita` con clave `hogar_id`, `fecha`.

Quedan: `hogar`, `visita`, `servicio`, `hogar_servicio`. **Cuatro tablas.**

Paso 3 — a tercera forma normal. Quedan tres dependencias transitivas, repartidas en las dos tablas nuevas. `localidad` → `provincia` saca `localidad` de `hogar`. `encuestador_legajo` → `encuestador_nombre`, `encuestador_telefono` saca `encuestador` de `visita`.

Quedan: `localidad`, `hogar`, `encuestador`, `visita`, `servicio`, `hogar_servicio`. **Seis tablas.**

Y la comparación, que es el punto de toda la sección:

Camino	Cómo se decidió	Resultado
Clase 2, conceptual	qué cosas existen, y cómo se vinculan	6 tablas
Clase 3, formal	qué dependencias hay, y de dónde salen	las mismas 6 tablas

Coinciden tabla por tabla, columna por columna y clave por clave. Eso no es una casualidad ni una comprobación decorativa: es la señal de que los dos métodos describen la misma cosa.

Y de ahí sale la conclusión práctica de las dos clases juntas:

El diseño conceptual y la normalización no son alternativas: son un método y su verificación. Se diseña conceptualmente, porque es la forma de hablar con quien conoce el negocio. Se verifica con las dependencias funcionales, porque es la forma de saber si quedó algo repetido adentro. Si los dos caminos no coinciden, hay algo que no se entendió, y casi siempre es una regla del negocio.

La última prueba. Cruzando las seis tablas se recupera la tabla plana original, con sus 45 filas y sus 12 columnas, valor por valor. Si no se recuperara, alguna descomposición estaría mal. El notebook lo verifica al final.

5. DESNORMALIZAR A PROPÓSITO

5.1 Qué se gana y qué se paga

Todo el taller empujó en una dirección. Ahora la parte incómoda: **normalizar tiene costos, y a veces conviene no hacerlo.**

El balance, sin adornos:

	Tabla plana	Seis tablas en 3FN
Cada hecho escrito	muchas veces	una vez
Cambiar un dato	tocar muchas filas	tocar una celda
Contradicciones internas	posibles	imposibles por construcción
Cargar algo que todavía nada referencia	no se puede	se puede
Ver todo junto	está a la vista	hay que cruzar seis tablas
Entender la estructura	una tabla	seis tablas y sus vínculos
Consultas grandes	rápidas	más lentas: el cruce cuesta

Las cuatro primeras filas son a favor de normalizar, y son las importantes. Las tres últimas son el precio, y es real.

Desnormalizar es la decisión de volver a juntar a propósito, aceptando la redundancia a cambio de comodidad o de velocidad. Es una decisión legítima. Lo que no es legítimo es no haber normalizado nunca y llamarlo desnormalización: una cosa es elegir el costo conociéndolo, y otra es no haberse enterado.

5.2 La vista: la tabla plana sin duplicar el dato

Antes de desnormalizar conviene saber que casi siempre no hace falta, porque hay una salida que da las dos cosas.

Una **vista** es una consulta guardada con nombre. Se define una vez —"cruzá estas seis tablas de esta manera"— y desde ahí se usa **como si fuera una tabla**. Se le pueden pedir filas, filtrar, contar, y quien la consulta no necesita saber que atrás hay seis tablas.

Lo que la hace valiosa es que **la vista no guarda datos**. Cada vez que se la consulta, se cruzan las tablas de nuevo. Así que:

- Se ve la tabla plana completa, con sus 45 filas y sus 12 columnas.
- El dato sigue estando escrito una sola vez.
- Si alguien corrige el teléfono de E04 en su única celda, la vista lo muestra corregido al instante.

Es la respuesta a "y entonces cómo hago para ver todo junto". Se hace una vista. En una base de datos es una línea, y se define una vez para siempre.

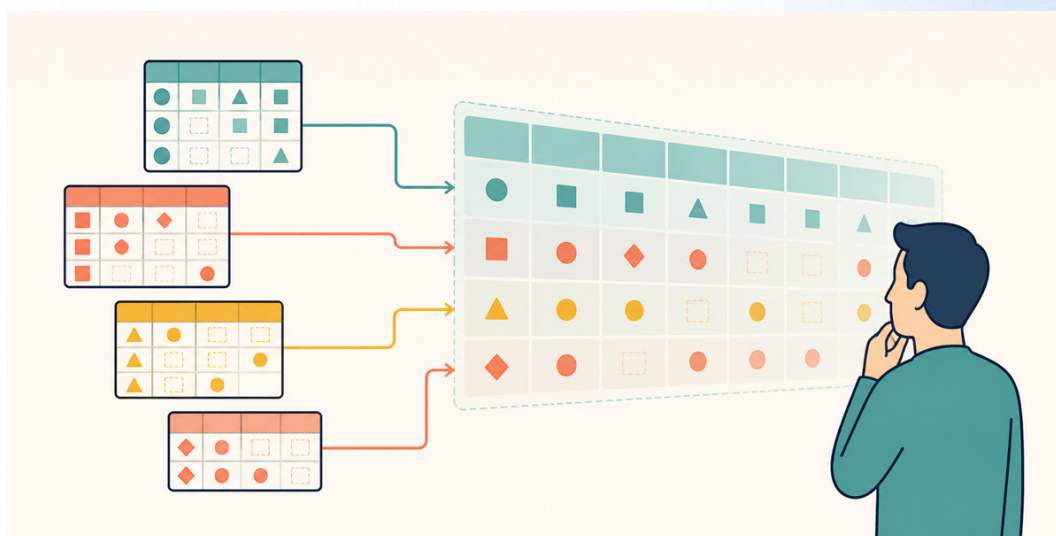


Figura 8. La vista muestra la tabla ancha y no la guarda. Es la comodidad sin la redundancia.

5.3 Cuándo se justifica desnormalizar

Con las vistas disponibles, quedan pocos casos, y conviene conocerlos porque son reales:

- **Cuando el cruce es demasiado caro y la consulta es muy frecuente.** En volúmenes grandes, cruzar seis tablas millones de veces por día cuesta. Ahí se guarda el resultado ya armado. Es una decisión de rendimiento, y se toma con mediciones, no con intuición.
- **Cuando el dato es histórico y no tiene que actualizarse.** Una factura emitida guarda el nombre y el domicilio del cliente **tal como estaban ese día**, a propósito. Si el cliente se muda, la factura vieja no tiene que cambiar. Acá la redundancia no es un defecto: es el requisito.
- **Cuando el destino es análisis y no operación.** Acá vuelve la palabra **tabla ancha**, en el sentido del T8001: un formato cómodo para analizar, no un diseño de base. Un tablero o un archivo para analizar suelen querer una tabla ancha, y está bien: no se cargan datos nuevos ahí, así que las tres anomalías no tienen dónde aparecer. Es exactamente el archivo con el que trabajó el T8001.

El tercer caso es el que más importa para este público, y conviene decirlo derecho: **la tabla plana del T8001 no estaba mal**. Para analizar, una tabla ancha es lo cómodo. El problema aparece cuando esa tabla ancha es también el lugar donde el dato se carga, se corrige y vive. Ahí es el diseño de la base, y ahí sí está mal.

5.4 Lista de verificación

Para revisar si una tabla está normalizada, cinco pasos en orden:

1. **¿Cuál es la clave?** Sin la clave no se puede evaluar nada, porque la 2FN y la 3FN se definen respecto de ella.
2. **¿Hay celdas con más de un valor, o columnas en plural?** Es 1FN. Se resuelve con una tabla nueva, no con más columnas.
3. **Si la clave es compuesta: ¿algún atributo depende de una sola parte?** Es 2FN. Probá tapando la otra mitad de la clave y preguntá si el valor sigue quedando fijado.
4. **¿Algún atributo depende de otro que no es clave?** Es 3FN. Buscá pares de columnas que se repitan siempre juntas.
5. **¿Cruzando las tablas se recupera la original?** Es la única prueba de que no se perdió información, y es la que más se saltea.

Y una advertencia sobre el paso 3 y el paso 4: las dependencias que encontrás en los datos son **candidatas**. Antes de separar una tabla, confirmá la regla con quien conoce el negocio. Separar por una dependencia que era una casualidad de la muestra deja un diseño que se rompe con la fila siguiente.

6. LOS EJEMPLOS EN GOOGLE COLAB

El notebook de esta clase sigue el orden del apunte, en seis bloques que vamos a ejecutar juntos:

1. **Buscar dependencias funcionales.** El procedimiento de la sección 2.3 sobre todos los pares de columnas, y la lista de candidatas que devuelve.
2. **Las candidatas que son casualidad.** provincia → localidad y encuestador_legajo → localidad, y por qué la primera pasa la prueba y es falsa.
3. **Una descomposición mala.** Cortar sin columna compartida y contar las filas inventadas que aparecen al cruzar.
4. **Los tres pasos.** De la tabla plana a seis tablas, mostrando después de cada paso qué dependencia quedó resuelta.
5. **La verificación contra la clase 2.** Comparar tabla por tabla el resultado de hoy con el esquema conceptual del apunte anterior.
6. **El cruce final, y la vista.** Reconstruir las 45 filas por 12 columnas, comprobar que son idénticas a las originales, y armar la vista que muestra la tabla plana sin guardarla.

No hay nada que completar: [03_dependencias_y_normalizacion.ipynb](#).